

COURSE OUTLINE

GitHub Actions — CI/CD Enablement & Migration

From Azure DevOps to GitHub Actions — intermediate to advanced track for AKS-based microservices teams

01 OVERVIEW

This programme upskills an existing Azure DevOps engineering team on GitHub Actions, in preparation for migrating CI/CD pipelines for a microservices application deployed to AKS. The team's current deployment layer uses Flux (GitOps) and continues to do so; this course is scoped strictly to GitHub Actions — the mechanics, patterns, and integration points needed to build, secure, and operate pipelines that hand off cleanly to the existing GitOps deployment flow. The programme is intermediate-to-advanced, built for engineers already fluent in CI/CD concepts and Git. It moves quickly through core mechanics and spends the majority of time on production concerns: pipeline design for a multi-microservice codebase, secrets and access control, environment protection, monitoring and troubleshooting, and AI-assisted workflow authoring using GitHub Copilot.

Delivery is four half-day sessions with instructor-led walkthroughs and guided labs — the instructor drives each build live and participants follow along on their own repositories where time permits, rather than working through fully independent lab exercises. Azure DevOps and Flux/GitOps are referenced only where needed for context or hand-off points; they are not taught in depth.

By the end of the programme, participants will be able to author, secure, and operate GitHub Actions workflows for containerised microservices, apply team-based access control, and use GitHub Copilot to accelerate workflow design and debugging.

02 PRE-REQUISITE KNOWLEDGE & AUDIENCE PROFILE

Participants should already be working CI/CD practitioners. This is not an introductory course — familiarity with Git branching/PR workflows, YAML, containers, and pipeline concepts (stages, jobs, variables, approvals) from Azure DevOps or an equivalent tool is assumed and will not be re-taught from first principles.

Working knowledge of Docker and basic Kubernetes/AKS concepts is expected, since the labs and examples are built around containerised microservices. General Azure familiarity (resource groups, container registries, identity/RBAC at a high level) is helpful for the access-control and OIDC discussion.

Target audience: DevOps and Platform Engineers, Build and Release Engineers, and Technical Leads currently on Azure DevOps who will design, own, or approve the migration of pipelines to GitHub Actions.

03 LAB SETUP & DURATION

GUIDED LAB ENVIRONMENT

Sessions are instructor-led and demo-driven; hands-on lab time is guided rather than mandatory independent exercise. Participants who want to follow along live will need:

- A GitHub account with permissions to create repositories and enable Actions in at least one org/repo
- GitHub Copilot licence (Business or Enterprise) enabled for the account being used
- VS Code with the GitHub Copilot and GitHub Actions extensions
- Git client (latest stable) and a modern browser with unrestricted access to github.com
- Docker Desktop, for container build examples
- Access to an Azure subscription and Container Registry (existing sandbox/dev subscription is fine) for the OIDC and registry-integration walkthroughs

Corporate proxy or firewall exceptions for github.com, the container registry, and Azure endpoints should be arranged before Session 1.

DURATION

— 4 Half-Days

~4 hours per session, 16 hours total. Instructor-led walkthroughs paired with guided, live-build labs.

DELIVERY MODE

Instructor-led training (ILT) — onsite, virtual, or hybrid. Guided labs; independent hands-on is optional.

LEVEL

Intermediate to Advanced.

LANGUAGE

English.

04 COURSE CONTENTS

Four half-day sessions, GitHub Actions-specific throughout. Azure DevOps and Flux/GitOps are referenced only at an overview or integration level, at the points where they hand off to or from GitHub Actions — they are not covered as standalone topics.

Session 1 (Half-Day) — Core Concepts & AI-Assisted Authoring

Fast-paced recap of GitHub Actions architecture for an already CI/CD-literate audience, moving quickly into real workflow authoring — with GitHub Copilot as a working tool from day one.

1. GitHub Actions Architecture (Accelerated)

- Workflow, event, job, step, runner — the execution model
- Workflow file structure and YAML syntax
- How GitHub Actions compares structurally to a stage/task-based pipeline model

2. Events & Triggers

- Push, pull request, and scheduled (cron) triggers
- Manual triggers (workflow_dispatch) and repository_dispatch
- Choosing triggers for build vs. release workflows

3. Jobs, Steps & Execution Control

- Defining jobs and steps; job dependencies with needs
- Parallel vs sequential execution
- GitHub-hosted vs self-hosted runners, and when self-hosted matters

4. Actions, Marketplace & Reuse

- Using pre-built Marketplace actions safely (pinning versions, trusted publishers)
- Overview of composite actions and reusable workflows (built on in Session 2)

5. AI-Assisted Workflow Design with GitHub Copilot

- Using Copilot Chat and inline suggestions to scaffold a new workflow from a plain-language description
- Prompting Copilot to explain an unfamiliar workflow or action
- Using Copilot to refactor a workflow (e.g., extract a step into a reusable action)
- Guardrails: reviewing and validating Copilot-generated YAML before merging

Session 2 (Half-Day) — Building CI Pipelines for Microservices

Building a real, production-shaped CI pipeline for a containerised microservice — the pattern participants will replicate across their own repositories.

6. Variables, Secrets & Contexts

- Environment variables vs. repository, environment, and organisation-level secrets
- Working with contexts (github, env, secrets, needs, matrix)

7. Workflow Control for Multi-Service Repos

- Conditional execution (if) for path-based / service-specific triggers
- Matrix builds — building and testing multiple microservices in one workflow
- Parallel jobs and timeout settings

8. Artifacts & Caching

- Uploading/downloading build artifacts between jobs
- Dependency caching for faster builds

9. Container Build Pipelines

- Building Docker images in a workflow
- Tagging strategy and pushing to a container registry
- Registry authentication approaches (overview — detailed in Session 3 via OIDC)

10. Standardising Pipelines: Reusable Workflows & Composite Actions

- Extracting a common build pattern into a reusable workflow
- When to use a composite action vs. a reusable workflow across many microservice repos

Session 3 (Half-Day) — Access Control, Security & Deployment Hand-off

The security and governance layer: securing the pipeline itself, controlling who can approve and deploy, and the clean hand-off point to the existing GitOps deployment flow.

11. Deployment Integration — Overview

- Where GitHub Actions' responsibility ends in a GitOps model (build → push image → update manifest reference)
- Integration point only — Flux reconciliation and AKS deployment mechanics are out of scope for this course

12. Keyless Cloud Authentication (OIDC)

- Why OIDC federation replaces long-lived cloud credentials in workflows
- Configuring OIDC trust between GitHub Actions and Azure

13. Environments & Approval Gates

- GitHub Environments — protection rules and required reviewers
- Modelling promotion across dev / staging / production environments

14. Team-Based Access Control

- Repository, environment, and organisation-scoped secrets and permissions
- Branch protection rules and required status checks
- CODEOWNERS for enforcing review ownership by team/service
- Mapping ADO service connections / variable groups to GitHub equivalents (overview level)

15. Securing the Workflow Itself

- Least-privilege GITHUB_TOKEN permissions
- Secure supply chain: pinning third-party actions, verified publishers
- Secret hygiene and avoiding accidental exposure in logs

Session 4 (Half-Day) — Monitoring, Troubleshooting & Applied Lab

Operating workflows day-to-day, using Copilot as a troubleshooting aid, and a consolidated guided lab tying the four sessions together.

16. Monitoring & Debugging

- Reading workflow logs and run history
- Diagnosing and re-running failed jobs, including matrix reruns
- Notifications and alerts on workflow status

17. Troubleshooting with GitHub Copilot

- Pasting a failed run's log into Copilot Chat to get a diagnosis and fix suggestion
- Using Copilot to propose a fix as a PR-ready YAML change

18. Advanced Authoring

- Dynamic/generated workflow patterns
- Using the GitHub API from within a workflow

19. Guided Lab — End-to-End Pipeline

- Instructor-led build of a complete pipeline: trigger → matrix build → container push → OIDC auth → environment-gated hand-off to GitOps
- Participants follow along on their own repository where environment access allows

20. Migration Approach & Wrap-Up

- High-level migration checklist for moving pipelines from Azure DevOps to GitHub Actions (overview level)
- Open Q&A against participants' real pipelines

Scope Notes

- Azure DevOps is referenced only for concept mapping (Session 1, Session 3) — no dedicated ADO module.
- Flux / GitOps mechanics are out of scope — Session 3 covers only the hand-off boundary (Actions builds and pushes; Flux takes over from there).
- Code quality checks (unit testing, linting, coverage) and Infrastructure-as-Code (Terraform) are excluded per programme scope.
- Deployment strategy modules (blue-green, canary, rolling) are excluded — these are owned by Flux/Flagger in this team's architecture, not GitHub Actions.

STALWART LEARNING · WHAT WE OFFER

- AI for Everyone, AI for Business Teams, AI for Leaders
- AI for Developers, AI for QA, AI for Analysts, AI for Security
- GitHub Copilot, MS Copilot for Office 365, Claude, Cursor, OpenAI Playground
- Cloud — AWS · Azure · GCP
- DevOps & Platform Engineering
- Data Engineering, Data Science & Analytics
- Cyber Security & AI Enablement
- Certification Vouchers